

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

Выполнил:

Андронов Денис

Группа

К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Д35: Реализация межсервисного взаимодействия посредством очередей сообщений

Срок: 15.04.2026

Задание:

- подключить и настроить rabbitMQ/kafka;
- реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Ход работы

Архитектура приложения

После доработки система состоит из следующих компонентов:

Компонент	Порт	Назначение
API Gateway	4000	Единая точка входа, проксирование HTTP-запросов
Auth Service	4001	Регистрация, аутентификация, RPC-проверка токена
Catalog Service	4002	Управление ресторанами, RPC-проверка ресторана
Booking Service	4003	Создание бронирований
Notification Service	-	Подписка на события о новых бронях
RabbitMQ	5672 / 15672	Брокер сообщений (AMQP / Management UI)

Компонент	Порт	Назначение
PostgreSQL	3000	Три отдельные БД: auth_db, catalog_db, booking_db

Клиент

|



API Gateway (:4000)

| — /api/auth/* → Auth Service (:4001)

| — /api/restaurants/* → Catalog Service (:4002)

| — /api/reservations/* → Booking Service (:4003)

Booking Service

| — RPC → очередь auth.validate.token → Auth Service

| — RPC → очередь catalog.restaurant.check → Catalog Service

| — Event → exchange reservation.events → Notification Service

Использованы два паттерна работы с очередями:

3.1. RPC (Request-Reply) - синхронный запрос через очередь

Применён для операций, где booking-service ожидает немедленный ответ:

Очередь	Producer	Consumer	Назначение
auth.validate.token	Booking Service	Auth Service	Проверка JWT-токена
catalog.restaurant.check	Booking Service	Catalog Service	Проверка ресторана

Механизм RPC:

1. Booking-service отправляет сообщение в очередь с полями correlationId и replyTo (временная очередь для ответа).
2. Auth/Catalog-service обрабатывает сообщение и отправляет результат в replyTo-очередь с тем же correlationId.
3. Booking-service получает ответ и продолжает обработку запроса.

Auth Service (authConsumer.ts) - consumer
очереди auth.validate.token:

- Принимает { token: string }
- Верифицирует JWT через jsonwebtoken
- Возвращает { isValid, userId, role }

Catalog Service (catalogConsumer.ts) - consumer
очереди catalog.restaurant.check:

- Принимает { restaurantId: number }
- Ищет ресторан в БД catalog_db

- Возвращает { found, id, name, isActive }

HTTP-эндпоинты /internal/validate и /internal/restaurants/:id/check, использовавшиеся в ДЗ4, удалены - их функциональность полностью перенесена на RabbitMQ.

Pub/Sub (Publish-Subscribe) - асинхронные события

Применён для уведомления о создании бронирования без блокировки основного потока:

Exchange	Тип	Routing Key	Publisher	Subscriber
reservation.events	topic	reservation.created	Booking Service	Notification Service

После успешного сохранения брони booking-service публикует событие:

```
{
  "reservationId": 2,
  "userId": 4,
  "restaurantId": 2,
  "date": "2026-05-27"
}
```

Notification Service подписан на routing key reservation.created и логирует уведомление:

[Notification] New reservation #2: user 4 booked restaurant 2 on 2026-05-27

Вывод

В рамках практической работы выполнена интеграция RabbitMQ в микросервисное приложение для бронирования столиков в ресторанах.

Достигнутые результаты:

1. **RabbitMQ подключён и настроен** - развёрнут через Docker Compose с Management UI для мониторинга очередей и exchange.
2. **Межсервисное взаимодействие переведено на очереди сообщений:**
 - синхронные HTTP-вызовы между booking, auth и catalog заменены на паттерн RPC через очереди;
 - добавлен асинхронный паттерн Pub/Sub для событий о создании бронирования.
3. **Добавлен API Gateway** - клиент взаимодействует с системой через единую точку входа (:4000), что упрощает интеграцию фронтенда и скрывает внутреннюю топологию сервисов.
4. **Добавлен Notification Service** - демонстрирует слабую связанность микросервисов: booking-service не знает о подписчиках на события и просто публикует сообщение.

Преимущества новой архитектуры:

- сервисы менее связаны между собой (loose coupling);
- возможность масштабирования consumers независимо;
- асинхронная обработка событий без блокировки основного потока;
- единая точка входа для клиентских запросов через API Gateway.